

Name: Phan Tran Thanh Huy

ID: ITCSIU22056

Lab 7

Lab Activity: Scheduling: Advanced: FCFS 1 Introduction to the FIRST COME FIRST SERVED (FCFS) Scheduling Algorithm

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Process	Balanced	IO Intense	Runtime (Milliseconds)
	☐	Process 1	☐	☐	30
	☒	Process 2	☐	☐	40
	☒	Process 3	☐	☐	50
	☒	Process 4	☐	☐	40
	☒	Process 5	☐	☐	30

System Configuration

Number of Processors: 1

IO Device Latency

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Turn OFF Icon Flashing

Scheduler Configuration

☐ Round Robin

☒ First Come First Served

☐ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

190

CPU Idle Time (Milliseconds)

0

Balanced processes perform IO every 15 ms of Runtime

IO-intensive processes perform IO every 7 ms of Runtime

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	30	0	0	0	30
Process 2	40	0	30	0	70
Process 3	50	0	70	0	120
Process 4	40	0	120	0	160
Process 5	30	0	160	0	190

Free Run

Pause

Reset Simulation

Done

Q1. What is the total execution time of the processes (how long between starting and finishing)?

Ans: The total execution time = 30 + 40 + 50 + 40 + 30 = 190ms

Q2. Do you think the FCFS scheduling algorithm is fair? Does fairness actually matter if all processes run for a very short time only? Try setting the process 1 Runtime to 300 milliseconds and repeat the experiment. Look at the wait time for each process – does this seem reasonable?

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
Process 1	☐	☐	☐	300
Process 2	☒	☐	☐	40
Process 3	☒	☐	☐	50
Process 4	☒	☐	☐	40
Process 5	☒	☐	☐	30

System Configuration

Number of Processors: 1

IO Device Latency

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Turn OFF Icon Flashing

Scheduler Configuration

☐ Round Robin

☒ First Come First Served

☐ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

460

CPU Idle Time (Milliseconds)

0

Balanced processes perform IO every 15 ms of Runtime

IO-intensive processes perform IO every 7 ms of Runtime

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	300	0	0	0	300
Process 2	40	0	300	0	340
Process 3	50	0	340	0	390
Process 4	40	0	390	0	430
Process 5	30	0	430	0	460

Free Run

Pause

Reset Simulation

Done

Ans: No FCFS is not very fair. Because FCFS suffers from the convoy effect. If the first process runs for a long time, all other processes must wait behind it, even if they are short.

Q3. Do you think the FCFS scheduling algorithm is efficient (how often is the CPU idle).

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
Process 1	☐	☒	☐	30
Process 2	☒	☐	☐	40
Process 3	☒	☐	☐	50
Process 4	☒	☐	☐	40
Process 5	☒	☐	☐	30

System Configuration

Number of Processors: 1

IO Device Latency

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Turn OFF Icon Flashing

Scheduler Configuration

☐ Round Robin
☒ First Come First Served
☐ Shortest Job First
☐ Shortest Remaining Job Next

Quantum Size

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

280

CPU Idle Time (Milliseconds)

90

Balanced processes perform IO every 15 ms of Runtime
IO-intensive processes perform IO every 7 ms of Runtime

Free Run

Pause

Reset Simulation

Done

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	30	0	0	10	40
Process 2	40	0	40	20	100
Process 3	50	0	100	30	180
Process 4	40	0	180	20	240
Process 5	30	0	240	10	280

Ans: when all processes are switched to Balanced, those perform some I/O, so they spend time in the Blocked state. However, FCFS is not very efficient when processes perform I/O, because the CPU may be idle while waiting for I/O to complete.

Q4. From your observations, do you think the FCFS algorithm is preemptive or non-preemptive?

Ans: The FCFS is a non-preemptive scheduling algorithm, because

- The quantum setting does not matter.
- The “convoy effect” happens.

Lab Activity: Scheduling: Advanced: SJF 1 Introduction to the SHORTEST JOB FIRST (SJF) Scheduling Algorithm

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
Process 1	☐	☐	☐	30
☒ Process 2	☐	☐	☐	40
☒ Process 3	☐	☐	☐	50
☒ Process 4	☐	☐	☐	40
☒ Process 5	☐	☐	☐	30

System Configuration

Number of Processors: 1

IO Device Latency

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Turn OFF Icon Flashing

Scheduler Configuration

☐ Round Robin
☐ First Come First Served
☒ Shortest Job First
☐ Shortest Remaining Job Next

Quantum Size

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

190

CPU Idle Time (Milliseconds)

0

Balanced processes perform IO every 15 ms of Runtime
IO-intensive processes perform IO every 7 ms of Runtime

Free Run

Pause

Reset Simulation

Done

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	30	0	0	0	30
Process 2	40	0	60	0	100
Process 3	50	0	140	0	190
Process 4	40	0	100	0	140
Process 5	30	0	30	0	60

Q1. What is the total execution time of the processes (how long between starting and finishing)?

Ans: The total execution time = $30 + 40 + 50 + 40 + 30 = 190\text{ms}$

Q2. Do you think the SJF scheduling algorithm is fair? Does fairness actually matter if all processes run for a very short time only? Try setting the process 1 Runtime to 300 milliseconds and repeat the experiment. Look at the wait time for each process – does this seem reasonable?

Scheduling Algorithms - Advanced

Process 1

☐ CPU Intense

☐ Balanced

☐ IO Intense

Runtime (Milliseconds)

Process 2

☒ CPU Intense

☐ Balanced

☐ IO Intense

Runtime (Milliseconds)

Process 3

☒ CPU Intense

☐ Balanced

☐ IO Intense

Runtime (Milliseconds)

Process 4

☒ CPU Intense

☐ Balanced

☐ IO Intense

Runtime (Milliseconds)

Process 5

☒ CPU Intense

☐ Balanced

☐ IO Intense

Runtime (Milliseconds)

RUN

READY

BLOCKED

COMPLETED

System Configuration

Number of Processors: 1

IO Device Latency

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Scheduler Configuration

☐ Round Robin
☐ First Come First Served
☒ Shortest Job First
☐ Shortest Remaining Job Next

Quantum Size

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

460

CPU Idle Time (Milliseconds)

0

Balanced processes perform IO every 15 ms of Runtime
 IO-intensive processes perform IO every 7 ms of Runtime

Turn OFF Icon Flashing

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	300	0	160	0	460
Process 2	40	0	30	0	70
Process 3	50	0	110	0	160
Process 4	40	0	70	0	110
Process 5	30	0	0	0	30

Free Run

Pause

Reset Simulation

Done

Ans: SJF is not fully fair. It *favors short jobs* and can delay long jobs. With P1 = 300ms, P1 waits until all the shorter processes finish and its waiting time becomes very large (about 160ms before it even starts). So SJF gives excellent performance for short jobs, but long jobs wait a long time. Fairness decreases when job lengths differ greatly.

Q3. Do you think the SJF scheduling algorithm is efficient (how often is the CPU idle).

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
 Process 1	☐	☒	☐	30
 Process 2	☒	☐	☐	40
 Process 3	☒	☐	☐	50
 Process 4	☒	☐	☐	40
 Process 5	☒	☐	☐	30

System Configuration

Number of Processors: 1

IO Device Latency

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Turn OFF Icon Flashing

Scheduler Configuration

☐ Round Robin

☐ First Come First Served

☒ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

280

CPU Idle Time (Milliseconds)

90

Balanced processes perform IO every 15 ms of Runtime

IO-intensive processes perform IO every 7 ms of Runtime

Free Run

Pause

Reset Simulation

Done

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	30	0	0	10	40
Process 2	40	0	80	20	140
Process 3	50	0	200	30	280
Process 4	40	0	140	20	200
Process 5	30	0	40	10	80

Ans: SJF is not specifically efficient when I/O is involved. CPU idle time still occurs whenever all Balanced processes are doing I/O at the same time.

Q4. From your observations, do you think the SJF algorithm is preemptive or non-preemptive?

Ans: SJF here is non-preemptive, because

- The running process keeps the CPU until it finishes or blocks for I/O.
- SJF only chooses the shortest job at the moment of selection, not during execution.

Q5. What differences can you identify between the behaviour of SJF and that of FCFS?

1. Order of execution:

- FCFS: first to arrive runs first
- SJF: shortest job runs first

2. Average waiting time:

- SJF gives lower waiting time
- FCFS can be bad if a long job arrives early (convoy effect)

3. Fairness:

- FCFS: fair, no starvation
- SJF: unfair to long jobs (possible starvation)

4. Responsiveness:

- SJF: better for short jobs
- FCFS: worse when jobs vary in length

5. Preemption: Both FCFS and this SJF are non-preemptive.

Lab Activity: Scheduling: Advanced: RR 1 Introduction to the ROUND ROBIN (RR) Scheduling Algorithm

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
Process 1	☒	☐	☐	30
Process 2	☒	☐	☐	40
Process 3	☒	☐	☐	50
Process 4	☒	☐	☐	40
Process 5	☒	☐	☐	30

System Configuration

Number of Processors: 1

IO Device Latency

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Scheduler Configuration

☒ Round Robin

☐ First Come First Served

☐ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

190

CPU Idle Time (Milliseconds)

0

Balanced processes perform IO every 15 ms of Runtime
IO-intensive processes perform IO every 7 ms of Runtime

Turn OFF Icon Flashing

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	30	0	80	0	110
Process 2	40	0	120	0	160
Process 3	50	0	140	0	190
Process 4	40	0	140	0	180
Process 5	30	0	120	0	150

Free Run

Pause

Reset Simulation

Done

Q1. What is the total execution time of the processes (how long between starting and finishing)?

Ans: All processes are CPU-intensive (no I/O), and in RR the CPU never becomes idle.

Total required CPU time = $30 + 40 + 50 + 40 + 30 = 190\text{ms}$.

Q2. Do you think the RR scheduling algorithm is fair? Does fairness actually matter if all processes run for a very short time only? Try setting the process 1

Runtime to 300 milliseconds and repeat the experiment. Look at the wait time for each process – does this seem reasonable?

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
Process 1	☒	☐	☐	300
Process 2	☒	☐	☐	40
Process 3	☒	☐	☐	50
Process 4	☒	☐	☐	40
Process 5	☒	☐	☐	30

System Configuration

Number of Processors: 1

IO Device Latency

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Scheduler Configuration

☒ Round Robin

☐ First Come First Served

☐ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

460

CPU Idle Time (Milliseconds)

0

Balanced processes perform IO every 15 ms of Runtime

IO-intensive processes perform IO every 7 ms of Runtime

Turn OFF Icon Flashing

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	300	0	160	0	460
Process 2	40	0	130	0	170
Process 3	50	0	160	0	210
Process 4	40	0	150	0	190
Process 5	30	0	120	0	150

Free Run

Pause

Reset Simulation

Done

Ans: Yes, RR is considered fair because every process gets equal CPU time (same quantum).

If P1 = 300ms:

- P1 still gets only 10ms per turn
- All other processes also get regular turns
- Nobody waits extremely long
- The wait times are much more even compared to FCFS or SJF

Q3. Do you think the RR scheduling algorithm is efficient (how often is the CPU idle).

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
Process 1	☐	☒	☐	30
☒ Process 2	☐	☒	☐	40
☒ Process 3	☐	☒	☐	50
☒ Process 4	☐	☒	☐	40
☒ Process 5	☐	☒	☐	30

System Configuration

Number of Processors: 1

IO Device Latency

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Turn OFF Icon Flashing

Scheduler Configuration

☒ Round Robin

☐ First Come First Served

☐ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

200

CPU Idle Time (Milliseconds)

10

Balanced processes perform IO every 15 ms of Runtime

IO-intensive processes perform IO every 7 ms of Runtime

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	30	0	80	10	120
Process 2	40	0	100	20	160
Process 3	50	0	120	30	200
Process 4	40	0	120	20	180
Process 5	30	0	110	10	150

Free Run

Pause

Reset Simulation

Done

Ans: RR is more efficient than FCFS or SJF with Balanced processes. Because when one process performs I/O, RR immediately switches to another READY process. Also, CPU idle time happens only when all processes are doing I/O at the same moment, which is rare.

Q4. From your observations, do you think the RR algorithm is preemptive or non-preemptive?

Ans: RR is preemptive., because the scheduler forces a context switch every time the quantum (10ms) expires, even if the process is not finished.

Q5. What differences can you identify between the behavior of RR and that of FCFS?

1. Average waiting time:

- RR: Balanced
- FCFS: Very high for later jobs

2. Fairness:

- FCFS: Unfair with long jobs
- RR: Fair

3. Responsiveness:

- RR: Very good
- FCFS: Very poor

4. Preemption:

- RR: Preemptive
- FCFS: Non-preemptive

5. Short job performance:

- RR: Not optimal
- FCFS: Depends on arrival order

6. Long job starvation:

- RR: Impossible
- FCFS: Possible for short job (convoy effect)

Q6. What differences can you identify between the behavior of RR and that of SJF?

1. Focus:

- RR: Equal sharing
- SJF: Job length optimization

2. Fairness:

- SJF: Unfair to long jobs
- RR: Very fair

3. Context switching:

- RR: High
- SJF: Low

4. Preemption:

- RR: Preemptive
- SJF: Usually non-preemptive (in this simulation)

5. Short job performance:

- RR: Not optimal
- SJF: Best average waiting time

Lab Activity: Scheduling: Advanced: SRJN 1 Introduction to the SHORTEST REMAINING JOB NEXT (SRJN) Scheduling Algorithm

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
Process 1	☒	☐	☐	30
Process 2	☒	☐	☐	40
Process 3	☒	☐	☐	50
Process 4	☒	☐	☐	40
Process 5	☒	☐	☐	30

System Configuration

Number of Processors: 1

IO Device Latency: 5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Turn OFF Icon Flashing

Scheduler Configuration

☐ Round Robin
☐ First Come First Served
☐ Shortest Job First
☒ Shortest Remaining Job Next

Quantum Size: 5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed: Fastest Slowest

System Statistics

Elapsed Time (Milliseconds): 190

CPU Idle Time (Milliseconds): 0

Balanced processes perform IO every 15 ms of Runtime
IO-intensive processes perform IO every 7 ms of Runtime

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	30	0	0	0	30
Process 2	40	0	60	0	100
Process 3	50	0	140	0	190
Process 4	40	0	100	0	140
Process 5	30	0	30	0	60

Free Run

Pause

Reset Simulation

Done

Q1. What is the total execution time of the processes (how long between starting and finishing)?

Ans: Since all processes are CPU-intensive and there is no I/O, the CPU is never idle.

Total CPU runtime = 30 + 40 + 50 + 40 + 30 = 190ms.

Because SRJN always picks the shortest remaining time, the order changes, but **total completion time remains 190 ms.**

Q2. Do you think the SRJN scheduling algorithm is fair? Does fairness actually matter if all processes run for a very short time only? Try setting the process 1 Runtime to 300 milliseconds and repeat the experiment. Look at the wait time for each process – does this seem reasonable?

Scheduling Algorithms - Advanced

The screenshot shows a process scheduling simulation interface. At the top left, the 'Process Configuration' table lists five processes with their respective settings. The 'Runtime State Display' shows the current state of the processes. The 'Runtime Statistics (Milliseconds)' table at the bottom left provides detailed timing data for each process. On the right, 'System Configuration' and 'Scheduler Configuration' panels allow for adjusting simulation parameters. The 'System Statistics' panel shows overall performance metrics.

Process	CPU Intense	IO Intense	Runtime (Milliseconds)
Process 1	☐	☐	300
Process 2	☒	☐	40
Process 3	☒	☐	50
Process 4	☒	☐	40
Process 5	☒	☐	30

Process	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	300	0	160	0	460
Process 2	40	0	30	0	70
Process 3	50	0	110	0	160
Process 4	40	0	70	0	110
Process 5	30	0	0	0	30

System Configuration	
Number of Processors:	1
IO Device Latency	5ms 10ms 15ms 20ms 25ms
Priority:	Fixed, all processes equal

Scheduler Configuration	
☐ Round Robin	
☐ First Come First Served	
☐ Shortest Job First	
☒ Shortest Remaining Job Next	
Quantum Size	5ms 10ms 15ms 20ms 25ms

System Statistics	
Elapsed Time (Milliseconds)	460
CPU Idle Time (Milliseconds)	0

Ans: SRJN is not fair, because it always chooses the job with the *smallest remaining* time, meaning:

- Long jobs can be postponed repeatedly
- Short jobs always jump ahead

If P1 = 300ms, all shorter jobs (30–50ms) constantly preempt it.
So Process 1 waits *a very long time*.

Q3. Do you think the SRJN scheduling algorithm is efficient (how often is the CPU idle).

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
Process 1	☐	☒	☐	30
Process 2	☒	☐	☐	40
Process 3	☒	☐	☐	50
Process 4	☒	☐	☐	40
Process 5	☒	☐	☐	30

System Configuration

Number of Processors: 1

IO Device Latency

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Turn OFF Icon Flashing

Scheduler Configuration

☐ Round Robin
☐ First Come First Served
☐ Shortest Job First
☒ Shortest Remaining Job Next

Quantum Size

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

215

CPU Idle Time (Milliseconds)

25

Balanced processes perform IO every 15 ms of Runtime
IO-intensive processes perform IO every 7 ms of Runtime

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	30	0	0	10	40
Process 2	40	0	65	20	125
Process 3	50	0	135	30	215
Process 4	40	0	100	20	160
Process 5	30	0	30	10	70

Free Run

Pause

Reset Simulation

Done

Ans: Yes, SRJN is very efficient because:

- It immediately switches to another ready job whenever one blocks for I/O
- CPU rarely goes idle unless *all* processes are waiting for I/O

Q4. From your observations, do you think the SRJN algorithm is preemptive or non-preemptive?

Ans: SRJN is preemptive. It interrupts the currently running job whenever a shorter job arrives.

Q5. What differences can you identify between the behavior of SRJN and that of FCFS?

SRJN:

- Preemptive
- Always picks shortest remaining time
- Short jobs finish very quickly
- Efficient but unfair
- Low average waiting time

FCFS:

- Non-preemptive
- Jobs run in arrival order
- Long jobs delay short jobs
- High waiting time
- Simple but inefficient

Q6. What differences can you identify between the behavior of SRJN and that of SJF?

SRJN:

- Preemptive
- Always picks shortest remaining time
- A shorter job can interrupt at any time
- Uses remaining job length
- Best response time

SJF:

- Non-preemptive
- Once a job starts, it runs to completion
- Uses full job length
- Worse response time

Q7. What differences can you identify between the behavior of SRJN and that of RR?

SRJN:

- Preemptive via job length comparison
- Unfair (short jobs dominate)
- Best responsiveness for short jobs
- Long jobs may starve
- Switching only when a shorter job arrives

RR:

- Preemptive via fixed quantum
- Fair (each job gets equal time)
- Good responsiveness
- Long jobs progress slowly but steadily
- Time-slice switching overhead

Lab Activity: Scheduling: Advanced: SRJN 2 Comparison of the SRJN and RR Scheduling Algorithms

Both SRJN and RR are preemptive. The objective of this activity is to compare the efficiency and fairness of the two algorithms.

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Process	Balanced	IO Intense	Runtime (Milliseconds)
	☐	Process 1	☐	☐	40
	☒	Process 2	☐	☐	40
	☒	Process 3	☐	☐	40
	☒	Process 4	☐	☒	30
	☒	Process 5	☐	☒	30

System Configuration

Number of Processors: 1

IO Device Latency

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Turn OFF Icon Flashing

Scheduler Configuration

☒ Round Robin

☐ First Come First Served

☐ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

188

CPU Idle Time (Milliseconds)

8

Balanced processes perform IO every 15 ms of Runtime

IO-intensive processes perform IO every 7 ms of Runtime

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	40	0	88	0	128
Process 2	40	0	105	0	145
Process 3	40	0	122	0	162
Process 4	30	0	111	40	181
Process 5	30	0	118	40	188

Free Run

Pause

Reset Simulation

Done

Scheduling Algorithms - Advanced

 Process 1 ☐ CPU Intense ☐ Balanced ☐ IO Intense ☐ Runtime (Milliseconds):

 ☒ Process 2 ☐ CPU Intense ☐ Balanced ☐ IO Intense ☐ Runtime (Milliseconds):

 ☒ Process 3 ☐ CPU Intense ☐ Balanced ☐ IO Intense ☐ Runtime (Milliseconds):

 ☒ Process 4 ☐ CPU Intense ☐ Balanced ☐ IO Intense ☐ Runtime (Milliseconds):

 ☒ Process 5 ☐ CPU Intense ☐ Balanced ☐ IO Intense ☐ Runtime (Milliseconds):

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	40	0	56	0	96
Process 2	40	0	100	0	140
Process 3	40	0	140	0	180
Process 4	30	0	28	40	98
Process 5	30	0	30	40	100

System Configuration

Number of Processors: 1

IO Device Latency:

Priority: Fixed, all processes equal

Scheduler Configuration

☐ Round Robin

☐ First Come First Served

☐ Shortest Job First

☒ Shortest Remaining Job Next

Quantum Size:

Animation Control

Simulation Speed:

System Statistics

Elapsed Time (Milliseconds):

CPU Idle Time (Milliseconds):

Balanced processes perform IO every 15 ms of Runtime
IO-intensive processes perform IO every 7 ms of Runtime

Turn OFF Icon Flashing

Free Run

Pause

Reset Simulation

Done

Q1. Which algorithm was more efficient in these simulations? Is this difference due to a carefully chosen set of processes, or will it apply generally?

Ans: SRJN is generally more efficient than RR, unless all processes have the same runtime. This is partly due to the chosen set of processes: we have a mix of long CPU-intensive processes and shorter IO-intensive processes. SRJN prioritizes short processes, reducing idle-time.

Q2. Which algorithm was fairest in these simulations? Is this difference due to a carefully chosen set of processes, or will it apply generally?

Ans: RR is fairer than SRJN (waiting times are more uniform). This is not just because of the chosen processes; it is inherent to the algorithms. RR inherently cycles fairly through processes, while SRJN favors short processes.

Q3. Which algorithm gave rise to the highest responsiveness of the processes in these simulations? Is this difference due to a carefully chosen set of processes, or will it apply generally?

Ans: SRJN provides higher responsiveness (shorter waiting times for short processes). This is partly due to the process set, since IO-intensive processes are shorter. SRJN prioritizes them, reducing their waiting and total time.

Lab Activity: Scheduling: Advanced: Quantum Size 1

Investigation of the effects of changing the size of the Scheduling Quantum

Scheduling Algorithms - Advanced

	CPU Intense	Process Configuration	IO Intense	Runtime (Milliseconds)
☐ Process 1	☐	☐	☐	40
☒ Process 2	☐	☒	☐	50
☒ Process 3	☐	☐	☒	40
☒ Process 4	☐	☒	☐	30
☒ Process 5	☐	☐	☒	30

System Configuration

Number of Processors: 1

IO Device Latency: 5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Turn OFF Icon Flashing

Scheduler Configuration

☒ Round Robin

☐ First Come First Served

☐ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size: 5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed: Fastest Slowest

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

System Statistics

Elapsed Time (Milliseconds): 198

CPU Idle Time (Milliseconds): 8

Balanced processes perform IO every 15 ms of Runtime

IO-intensive processes perform IO every 7 ms of Runtime

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	40	0	100	0	140
Process 2	50	0	101	30	181
Process 3	40	0	108	50	198
Process 4	30	0	95	10	135
Process 5	30	0	118	40	188

Free Run

Pause

Reset Simulation

Done

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
 Process 1	☐	☐	☐	40
☒  Process 2	☐	☒	☐	50
☒  Process 3	☐	☐	☒	40
☒  Process 4	☐	☐	☐	30
☒  Process 5	☐	☐	☒	30

System Configuration

Number of Processors: 1

IO Device Latency:

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Scheduler Configuration

☒ Round Robin

☐ First Come First Served

☐ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size:

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed:

Fastest Slowest

Runtime State Display

RUN: 

READY: 

BLOCKED: 

COMPLETED:  

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	40	0	75	0	115
Process 2	50	0	113	30	193
Process 3	40	0	98	50	188
Process 4	30	0	109	10	149
Process 5	30	0	110	40	180

System Statistics

Elapsed Time (Milliseconds):

CPU Idle Time (Milliseconds):

Balanced processes perform IO every 15 ms of Runtime

IO-intensive processes perform IO every 7 ms of Runtime

Free Run Pause Reset Simulation Done

Turn OFF Icon Flashing

Scheduling Algorithms - Advanced

Process Configuration

CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
☐	☒	☐	40
☒	☐	☐	50
☒	☐	☐	40
☒	☐	☐	30
☒	☐	☐	30

Runtime State Display

READY

BLOCKED

RUN



System Configuration

Number of Processors: 1

IO Device Latency: _____

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Scheduler Configuration

☒ Round Robin

☐ First Come First Served

☐ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size: _____

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed: _____

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds) [201]

CPU Idle Time (Milliseconds) [11]

Balanced processes perform IO every 15 ms of Runtime
 IO-intensive processes perform IO every 7 ms of Runtime

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	40	0	88	0	128
Process 2	50	0	82	30	162
Process 3	40	0	111	50	201
Process 4	30	0	71	10	111
Process 5	30	0	118	40	188

Free Run
Pause
Reset Simulation

Done

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
Process 1	☐	☐	☐	40
☒ Process 2	☐	☒	☐	50
☒ Process 3	☐	☐	☒	40
☒ Process 4	☐	☒	☐	30
☒ Process 5	☐	☐	☒	30

System Configuration

Number of Processors: 1

IO Device Latency

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Turn OFF Icon Flashing

Scheduler Configuration

☒ Round Robin

☐ First Come First Served

☐ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size

5ms 10ms 15ms 20ms 25ms

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Animation Control

Simulation Speed

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

CPU Idle Time (Milliseconds)

Balanced processes perform IO every 15 ms of Runtime

IO-intensive processes perform IO every 7 ms of Runtime

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	40	0	44	0	84
Process 2	50	0	82	30	162
Process 3	40	0	111	50	201
Process 4	30	0	81	10	121
Process 5	30	0	118	40	188

Free Run

Pause

Reset Simulation

Done

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
Process 1	☐	☐	☐	40
☒ Process 2	☐	☒	☐	50
☒ Process 3	☐	☐	☒	40
☒ Process 4	☐	☒	☐	30
☒ Process 5	☐	☐	☒	30

System Configuration

Number of Processors: 1

IO Device Latency

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Scheduler Configuration

☒ Round Robin

☐ First Come First Served

☐ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

201

CPU Idle Time (Milliseconds)

11

Balanced processes perform IO every 15 ms of Runtime
IO-intensive processes perform IO every 7 ms of Runtime

Turn OFF Icon Flashing

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	40	0	44	0	84
Process 2	50	0	82	30	162
Process 3	40	0	111	50	201
Process 4	30	0	81	10	121
Process 5	30	0	118	40	188

Free Run

Pause

Reset Simulation

Done

Q1. How does the quantum size affect fairness (i.e. which quantum size causes the processes to encounter similar wait times, and which quantum size causes the greatest difference in the wait times)? Why is this so?

Ans:

- **Small quantum (e.g., 5ms):** Processes experience similar waiting times → fairness is high.
- **Large quantum (e.g., 25ms):** Long processes can monopolize the CPU → short processes wait longer → greatest difference in waiting times → fairness decreases.
- **Reason:** Smaller quantum increases preemption and rotation frequency, distributing CPU time more evenly. Larger quantum reduces preemption, favoring longer processes.

Q2. How does the quantum size affect the mean wait-time? Why is this?

Ans:

- **Small quantum:** Mean waiting-time may be slightly higher due to context switching overhead.
- **Moderate quantum (10–15ms):** Mean waiting-time is minimized and balanced.
- **Large quantum:** Mean waiting-time increases for short processes as they wait for longer processes to finish their quantum.
- **Reason:** Smaller quantum allows fairer CPU sharing but adds overhead; larger quantum reduces overhead but delays short processes.

Q3. How does the quantum size affect the mean total-time-in-the-system? Why is this?

Ans:

- **Small quantum:** Slightly higher total-time-in-system due to context switching overhead.
- **Moderate quantum:** Balanced total-time-in-system.

- ## Lab Activity: Scheduling: Advanced: IO Latency 1 Investigation of the effects of changing the IO device latency

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
Process 1	☐	☐	☐	40
☒ Process 2	☐	☒	☐	50
☒ Process 3	☐	☐	☒	40
☒ Process 4	☐	☒	☐	30
☒ Process 5	☐	☐	☒	30

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	40	0	98	0	138
Process 2	50	0	82	45	177
Process 3	40	0	98	75	213
Process 4	30	0	83	15	128
Process 5	30	0	101	60	191

System Configuration

Number of Processors: 1

IO Device Latency

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Scheduler Configuration

☒ Round Robin

☐ First Come First Served

☐ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

CPU Idle Time (Milliseconds)

Balanced processes perform IO every 15 ms of Runtime
IO-intensive processes perform IO every 7 ms of Runtime

Turn OFF Icon Flashing

Free Run
Pause
Reset Simulation
Done

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Process Balanced	IO Intense	Runtime (Milliseconds)
Process 1	☐	☐	☐	40
☒ Process 2	☐	☒	☐	50
☒ Process 3	☐	☐	☒	40
☒ Process 4	☐	☒	☐	30
☒ Process 5	☐	☐	☒	30

System Configuration

Number of Processors:

IO Device Latency:

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Scheduler Configuration

☒ Round Robin

☐ First Come First Served

☐ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size:

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed:

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds):

CPU Idle Time (Milliseconds):

Balanced processes perform IO every 15 ms of Runtime
IO-intensive processes perform IO every 7 ms of Runtime

Turn OFF Icon Flashing

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	40	0	78	0	118
Process 2	50	0	96	45	191
Process 3	40	0	83	75	198
Process 4	30	0	97	15	142
Process 5	30	0	93	60	183

Free Run

Pause

Reset Simulation

Done

Scheduling Algorithms - Advanced

Process Configuration

CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
☐	☒	☐	40
☒	☐	☐	50
☒	☐	☐	40
☒	☐	☐	30
☒	☐	☐	30

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

System Configuration

Number of Processors: 1

IO Device Latency

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Scheduler Configuration

- ☒ Round Robin
- ☐ First Come First Served
- ☐ Shortest Job First
- ☐ Shortest Remaining Job Next

Quantum Size

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

214

CPU Idle Time (Milliseconds)

24

Balanced processes perform IO every 15 ms of Runtime
 IO-intensive processes perform IO every 7 ms of Runtime

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	40	0	81	0	121
Process 2	50	0	68	45	163
Process 3	40	0	99	75	214
Process 4	30	0	66	15	111
Process 5	30	0	106	60	196

Free Run
Pause
Reset Simulation
Done

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Process Balanced	IO Intense	Runtime (Milliseconds)
Process 1	☐	☐	☐	40
☒ Process 2	☐	☒	☐	50
☒ Process 3	☐	☐	☒	40
☒ Process 4	☐	☒	☐	30
☒ Process 5	☐	☐	☒	30

System Configuration

Number of Processors: 1

IO Device Latency: 5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Scheduler Configuration

☒ Round Robin

☐ First Come First Served

☐ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size: 5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed: Fastest Slowest

System Statistics

Elapsed Time (Milliseconds): 214

CPU Idle Time (Milliseconds): 24

Balanced processes perform IO every 15 ms of Runtime
IO-intensive processes perform IO every 7 ms of Runtime

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	40	0	44	0	84
Process 2	50	0	68	45	163
Process 3	40	0	99	75	214
Process 4	30	0	76	15	121
Process 5	30	0	106	60	196

Free Run

Pause

Reset Simulation

Done

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
Process 1	☐	☐	☐	40
☒ Process 2	☐	☐	☐	50
☒ Process 3	☐	☐	☐	40
☒ Process 4	☐	☐	☐	30
☒ Process 5	☐	☐	☐	30

System Configuration

Number of Processors: 1

IO Device Latency:

5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Scheduler Configuration

☒ Round Robin

☐ First Come First Served

☐ Shortest Job First

☐ Shortest Remaining Job Next

Quantum Size:

5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed:

Fastest Slowest

System Statistics

Elapsed Time (Milliseconds):

CPU Idle Time (Milliseconds):

Balanced processes perform IO every 15 ms of Runtime

IO-intensive processes perform IO every 7 ms of Runtime

Runtime State Display

RUN:

READY:

BLOCKED:

COMPLETED:

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	40	0	44	0	84
Process 2	50	0	68	45	163
Process 3	40	0	99	75	214
Process 4	30	0	76	15	121
Process 5	30	0	106	60	196

Free Run

Pause

Reset Simulation

Done

Q1. As IO device latency increases what happens to the total-time-in-the-system (the time it takes to execute to completion) of each process?

Do any of the processes take longer to execute? If so, why?

Do any of the processes take less time to execute? If so, why?

Ans:

- As IO device latency increases, IO-intensive and balanced processes take longer to complete because they spend more time in the BLOCKED state waiting for I/O operations.
- CPU-intensive processes are largely unaffected because they do little or no I/O.
- No processes take less time to execute; only CPU-intensive processes maintain similar total times, while processes performing I/O are delayed.

Q2. How does the IO device latency affect the mean total-time-in-the-system? Why is this? Is this what you expected?

Ans:

- Increasing IO latency increases the mean total-time-in-the-system.
- **Reason:** Processes that perform I/O (balanced and IO-intensive) spend more time waiting for I/O completion, which adds to their total system time.
- **Expectation:** Yes, this is expected because slower I/O devices delay processes that depend on them.

Q3. How does the IO device latency affect the mean wait-time? Why is this (think carefully)? Is this what you expected?

Ans:

- Increasing IO latency also increases mean wait-time.

- **Reason:** While a process waits for I/O, it may cause other processes to wait longer in the READY queue, especially if the CPU is idle waiting for I/O to complete.
- **Expectation:** Yes, this is expected because higher I/O latency slows down process progress, indirectly increasing waiting times.

Lab Activity: Scheduling: Advanced: Predicting Behaviour 1

Predict and Analyse the behaviour of the scheduling algorithms under various circumstances

Q1. Predict which process will finish first?

Ans: Process 3 (IO Intensive, 30ms) because SJF selects the process with the shortest total CPU runtime first.

Q2. Predict which process will finish last?

Ans: Process 2 (Balanced, 50ms) because it has the longest CPU runtime.

Q3. Predict how much time process 2 will spend in the Blocked State?

Ans: Process 2 is Balanced, so it does some I/O. The exact blocked time depends on the number of I/O operations and IO latency (10ms), but it will spend a moderate amount of time in the Blocked state—less than P3, more than P1 (which is CPU-intensive and does no I/O).

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
Process 1	☐	☐	☐	40
☒ Process 2	☐	☒	☐	50
☒ Process 3	☐	☐	☒	30
☐ Process 4	☐	☒	☐	30
☐ Process 5	☐	☐	☒	30

System Configuration

Number of Processors:

IO Device Latency

 5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Scheduler Configuration

☐ Round Robin
☐ First Come First Served
☒ Shortest Job First
☐ Shortest Remaining Job Next

Quantum Size

 5ms 10ms 15ms 20ms 25ms

Turn OFF Icon Flashing

Animation Control

Simulation Speed

 Fastest Slowest

System Statistics

Elapsed Time (Milliseconds)

CPU Idle Time (Milliseconds)

Balanced processes perform IO every 15 ms of Runtime
 IO-intensive processes perform IO every 7 ms of Runtime

Runtime State Display

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	40	0	70	0	110
Process 2	50	0	110	30	190
Process 3	30	0	0	40	70
Process 4	0	0	0	0	0
Process 5	0	0	0	0	0

Free Run

Pause

Reset Simulation

Done

Q4. Predict which process will finish first?

Ans: Process 3 (IO Intensive, 30ms) because SRJN always picks the process with the shortest remaining time, and P3 starts with the smallest total runtime.

Q5. Predict which process will finish last?

Ans: Process 2 (Balanced, 50ms) because it has the longest remaining runtime and may be preempted by shorter processes, delaying its completion.

Q6. Predict how much time process 2 will spend in the Blocked State?

[illegible]

Lab Activity: Scheduling: Advanced: Predicting Behaviour 2
Predict and Analyse the behaviour of the scheduling algorithms under various circumstances

Q1. Predict which process will finish first?

Ans: Process 3 (IO Intensive, 30ms) because SRJN always selects the process with the shortest remaining runtime.

Q2. Predict which process will finish last?

Ans: Process 2 (Balanced, 50ms) because It has the longest runtime and may be preempted by shorter processes.

Q3. Predict how much time process 1 will spend in the Blocked State?

Ans: Process 1 is Balanced, so it does some I/O and spends moderate time in Blocked state. With IO latency = 20ms, total blocked time will be proportional to the number of I/O operations.

Q4. Predict which process will finish first?

Ans: Process 3 (IO Intensive, 30ms) because shortest remaining CPU time is still P3; preemptive SRJN favors shortest jobs.

Q5. Predict which process will finish last?

Ans: Process 1 (CPU Intensive, 70ms) because of longest runtime, no I/O to give it a chance to be blocked; SRJN allows shorter jobs to preempt it.

Q6. Predict how much time process 1 will spend in the Blocked State?

Ans: 0ms, due to CPU-intensive process does no I/O.

Scheduling Algorithms - Advanced

The screenshot shows a complex simulation interface for scheduling algorithms. It features several interactive elements like sliders, checkboxes, and input fields, along with real-time statistics and state displays.

Process Configuration

	CPU Intense	Process	Balanced	IO Intense	Runtime (Milliseconds)
	☐	Process 1	☒	☐	70
	☒	Process 2	☒	☐	50
	☒	Process 3	☐	☒	30
	☐	Process 4	☒	☐	30
	☐	Process 5	☐	☒	30

Runtime State Display

Visual representation of process states:

- RUN:** A large yellow rectangle representing the running process.
- READY:** A yellow rectangle representing processes ready to run.
- BLOCKED:** A yellow rectangle representing blocked processes.
- COMPLETED:** Three colored circles (red, blue, black) representing finished processes.

System Configuration

Number of Processors: 1
 IO Device Latency: [Slider from 5ms to 25ms]
 Priority: Fixed, all processes equal

Scheduler Configuration

☐ Round Robin
☐ First Come First Served
☐ Shortest Job First
☒ Shortest Remaining Job Next

Quantum Size: [Slider from 5ms to 25ms]

Animation Control

Simulation Speed: [Slider from Fastest to Slowest]

System Statistics

Elapsed Time (Milliseconds): 183
 CPU Idle Time (Milliseconds): 33

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	70	0	33	80	183
Process 2	50	0	18	60	128
Process 3	30	0	10	80	120
Process 4	0	0	0	0	0
Process 5	0	0	0	0	0

Balanced processes perform IO every 15 ms of Runtime
 IO-intensive processes perform IO every 7 ms of Runtime

Buttons at the bottom: Free Run, Pause, Reset Simulation, Done

Lab Activity: Scheduling: Advanced: Predicting Behaviour 3
Predict and Analyse the behaviour of the scheduling algorithms under various circumstances

Q1. Predict which process will finish first?

Ans: Process 4 (IO Intensive, 30ms) because SRJN selects the process with the shortest remaining runtime first.

Q2. Predict which process will finish first?

Ans: Process 4 (30ms) because shortest job will complete earliest.

Q3. Predict which process will finish last?

Ans: Process 3 (CPU Intensive, 100ms) because of longest runtime, no I/O to allow preemption, delayed by shorter jobs.

Q4. Predict how much time process 1 will spend in the Blocked State?

Ans: Process 1 is IO-intensive, so it will spend significant time in Blocked state, roughly proportional to its number of I/O operations × IO latency (25ms).

Q5. Predict how much time process 4 will spend in the Run State?

Ans: Approximately equal to its CPU runtime (30 ms) since it is short and finishes quickly.

Scheduling Algorithms - Advanced

Process Configuration

	CPU Intense	Balanced	IO Intense	Runtime (Milliseconds)
☐ Process 1	☒	☐	☐	40
☒ Process 2	☐	☐	☐	50
☒ Process 3	☐	☐	☐	100
☒ Process 4	☐	☐	☐	30
☒ Process 5	☐	☐	☐	40

System Configuration

Number of Processors: 1

IO Device Latency: 5ms 10ms 15ms 20ms 25ms

Priority: Fixed, all processes equal

Turn OFF Icon Flashing

Scheduler Configuration

☐ Round Robin

☐ First Come First Served

☐ Shortest Job First

☒ Shortest Remaining Job Next

Quantum Size: 5ms 10ms 15ms 20ms 25ms

Animation Control

Simulation Speed: Fastest Slowest

System Statistics

Elapsed Time (Milliseconds): 350

CPU Idle Time (Milliseconds): 90

Balanced processes perform IO every 15 ms of Runtime

IO-intensive processes perform IO every 7 ms of Runtime

Runtime State Display

RUN

READY

BLOCKED

COMPLETED

Runtime Statistics (Milliseconds)

	Runtime Used	Runtime Remaining	Wait time	IO Time (Blocked)	Total time in system
Process 1	40	0	79	125	244
Process 2	50	0	125	175	350
Process 3	100	0	105	0	205
Process 4	30	0	77	100	207
Process 5	40	0	86	125	251

Free Run

Pause

Reset Simulation

Done

Q6. Predict which process will finish first?

Ans: Process 4 (IO Intensive, 30ms) because it still shortest remaining job.

Q7. Predict which process will finish first?

Ans: Process 4 (30ms) because of shortest CPU runtime, finishes earliest.

Q8. Predict which process will finish last?

Ans: Process 3 (CPU Intensive, 100ms) because of longest CPU runtime, preempted by shorter jobs.

Q9. Predict how much time process 2 will spend in the Blocked State?

Ans: Process 2 is IO-intensive, so it will spend moderate to significant time in Blocked state, roughly proportional to its I/O operations × new IO latency (20ms), slightly less than before.

Q10. Predict how much time process 3 will spend in the Run State?

Ans: It is 100ms, due to CPU-intensive process runs continuously when selected, total run time equals its CPU runtime.

